Design of a PD Controller for a Competitive Line Follower Robot

Mikel Dudi, Christos Angelakopoulos

Department of Electrical & Computer Engineering, University of Peloponnese

Patras, Greece

Abstract

This paper describes the development of a competitive Line Follower Robot and the optimization of its controller for racing scenarios, as developed by UoP Robotics, the University of Peloponnese's Robotic Team. It describes in detail the mathematical foundations, hardware, and electronics used in the prototype's construction. Additionally, the paper incorporates a MATLAB simulation of the Line follower Robot, providing a virtual platform to analyze and validate the robot's behavior and controller efficiency without setting up the real-life prototype and a course. The combination of theoretical and practical approaches aims to contribute to high-speed, reliable line-following robotics for competitive applications.

1. *Introduction*

2. *What is a PID Controller?*
   2.1. *Components of a PID Controller*
   2.2. *General PID Control Formula and Idea:*
   2.3. *Why a PD Controller?*
3. *System Design*
   3.1. *Circuit Design*
   3.2. *Hardware Configuration*
   3.3. *Software Architecture*
   3.4. *Practical Implementation using Arduino IDE*

MATLAB simulation and comments (second half)

4. *Introduction*
5. *System model*
   5.1. *Inputs to the System*
   5.2. *Output of the System*
6. *PD Controller Design*
   6.1. *Tracking error*
   6.2. *Control signal*
7. *Motor Control (Voltage Calculation and Visualization)*
8. *Real-Time Path Tracking*
9. *System Stability and Tuning and Key Takeaways*
10. *References and Sources*

1. Introduction

This paper presents the development of a line-following robot, LineScout L1, designed and built by the **UoP Robotics Team** for the **Robochallenge 2024** competition in Bucharest, Romania.

Line-following robots are autonomous systems that navigate a predefined track marked by a dark colored line on a light color surface. The objective of the competition is to complete the track in the shortest possible time while maintaining precise adherence to the line, even though sharp curves and varying radii. The fastest robot that successfully finishes the track is declared the winner.

While line-following robots may appear simple, their design and operation involve significant challenges. Tracks in competitions are unique, incorporating sharp angles, large and small radius curves, and variable path widths, all of which test the robot's ability to maintain stability and speed. The dark colored line is typically made of electrical tape, adding variability in adhesion and texture, which can further complicate line detection.

The UoP Robotics Team's robot uses a sensor to detect the position of the line, a motor driver for motor control, and a microcontroller to process sensor data and execute the control logic. The robot is powered by a pair of DC Motors, and its frame is PCB (Printed Circuit Board) for lightweight durability and optimal sensor alignment. A Proportional-Derivative (PD) controller is employed to adjust steering and speed dynamically, ensuring that the robot follows the line accurately while maximizing speed.

Additionally, robots in this category must adhere to strict rules: they must remain fully autonomous, conform to size constraints (250 mm × 250 mm × 200 mm), and cannot be modified or reprogrammed during the competition.

Control strategies for the robot are designed to handle both the direction and speed of the motors. The PD controller is used for smooth corrections to keep the robot on the track, while adaptive motor speed control allows for efficient navigation through varying track conditions. For instance, the robot slows down on tighter curves to avoid skidding and accelerates on straighter sections for optimal time performance.

This paper explores the development process, highlighting the hardware and software design choices, as well as the challenges encountered in optimizing the robot's performance for competitive conditions. Through careful sensor integration, motor control, and parameter tuning, the UoP Robotics Team developed a robot capable of achieving precision and speed in a demanding environment.

2. What is a PID Controller?

A PID Controller is a type of feedback control system widely used in automation and control systems to maintain a desired output (setpoint) by adjusting the system's input. The PID controller continuously calculates an error value as the difference between the desired setpoint and the measured process variable, and it applies a correction based on three key components: Proportional (P), Integral (I), and Derivative (D). The combination of these three terms enables a PID controller to achieve fast, stable, and accurate control for a variety of systems, ranging from robotics and motor control to industrial automation and process control. Its simplicity and effectiveness make it a cornerstone of modern control engineering.

2.1. Components of a PID Controller

Proportional (P):

The **Proportional** component is directly related to the current error. It determines the correction based on the magnitude of the error.

The correction is proportional to the error, meaning if the error is large, the correction will also be large. Conversely, if the error is small, the correction will be smaller.

Formula: $P = Kp \times Error$, **where:**

Kp is the proportional gain and

Error is the difference between the desired setpoint and the current value.

Integral (I):

The **Integral** component accounts for the accumulated past errors. Over time, even small errors can build up and cause drift, so the integral component helps correct this by integrating past error values.

It ensures that the system eliminates steady-state error (the error that persists over time), correcting for offsets that might not be addressed by the proportional control alone.

Formula: $I = Ki \times \int_{o}^{t} e(\tau)\, d\tau$, **where:**

Ki is the integral gain, and the

Integral term is the sum of past errors over time.

Derivative (D):

The **Derivative** component predicts future errors by looking at the rate of change of the error. If the error is changing rapidly, the derivative term helps apply a correction to counteract that change.

It helps reduce overshoot and oscillation by damping the system's response.

Formula: $D = Kd * \frac{de(t)}{dt}$ **where:**

Kd is the derivative gain, and the

Derivative term represents the rate of change of the error.

2.2. General PID Control Formula and Idea:

The overall output of a PID controller is the sum of these three components:

$$u(t) = P + I + D$$

Where:

- **P** is the proportional term.
- **I** is the integral term.
- **D** is the derivative term.

Or

$$u(t) = Kp * e(t) + Ki * \int_0^t e(\tau)\, d\tau + Kd * \frac{de(t)}{dt}$$

PID controllers are essential, in any serious, or competitive application that utilizes a control system. This type of control system helps maintain a desired output by minimizing the difference between a target value that is predefined (setpoint, SP) and actual system output (process variable, PV). It accomplishes this through these three key components: proportional, integral, and derivative control.

The proportional component responds to the current error, immediately correcting deviations from the setpoint. The integral component addresses cumulative errors over time, ensuring long-term inconsistencies are eliminated. The derivative component "predicts future trends in error" by analyzing the rate of change, damping to minimize oscillations.

By adjusting its output based on these three elements, a PID controller effectively balances speed, accuracy, and stability. This makes it highly versatile for applications such as temperature control, robotics, and process automation. Its main objective is to keep the system as close to the setpoint as possible while reducing oscillations, overshoot, and settling time.

2.3. Why a PD Controller?

The use of a PID control system for a line-following robot offers greater precision and adaptability compared to other methods, like an on-off control system, especially in dynamic environments, like following a non-straight, complex line. An on-off system works by switching the motors fully on or off depending on whether the robot is on or off the line. While easy to implement, this approach leads to jerky movements and frequent oscillations making it unsuitable for scenarios requiring smooth, high-speed navigation, such as competitive line-following. A PID controller continuously adjusts the robot's movement by calculating the proportional, integral, and derivative responses to the error (deviation from the line). This allows the robot to correct its path smoothly and efficiently, reducing overshoot and oscillations.

However, when it comes to competitive Line-following robots, the integral term makes hardly any important improvement on the robot’s control and movement. As was the case in this competition, the objective is for a robot to finish a track as fast as possible, and the integral term of the PID controller, not only does not help with this objective, but also hinders it. Working on accumulated error, it responds more slowly, having to make a more complex computation, compared to the proportional or derivative terms. This can reduce the robot's agility, which is critical in sharp turns or rapid path corrections.

Another reason for this omission was the hardware limitations of the motor driver, sensors and motors used. A steady-state error could be introduced by any of the components, thus making the integral term unnecessary. Skipping it did not impact the performance of the robot, at least to a degree that was even visible, so it was omitted altogether.

3. System Design

3.1. Circuit Design

In order to design the PCB(Printed Circuit Board) we used EasyEDA (Fig.1)

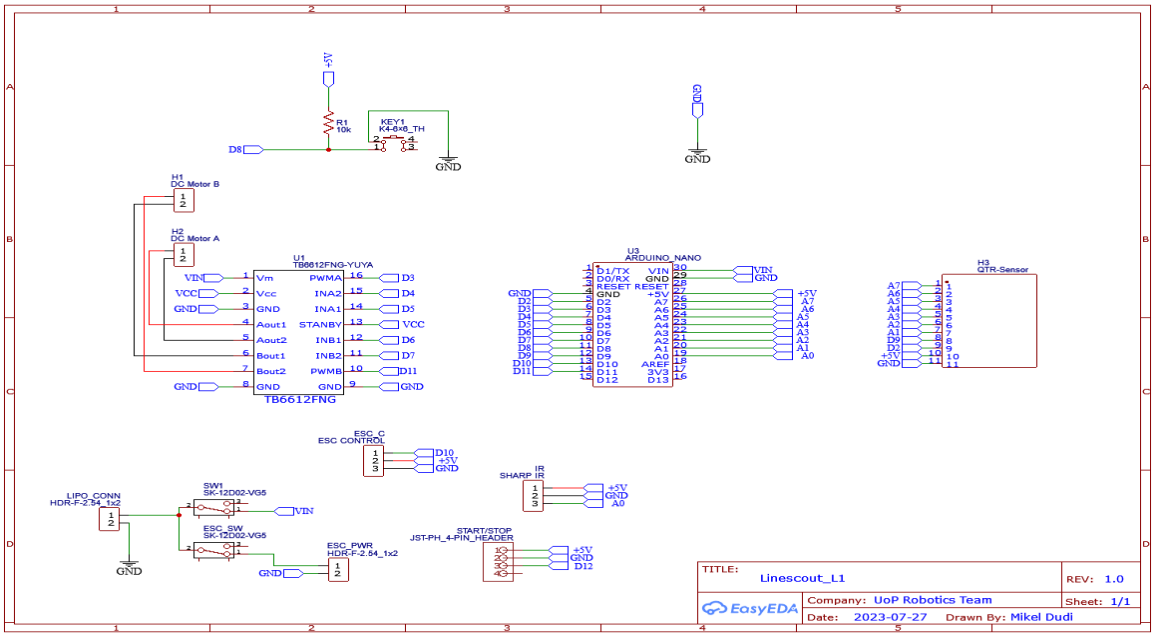


Figure 1. Line follower Circuit Diagram

3.2. Hardware Configuration

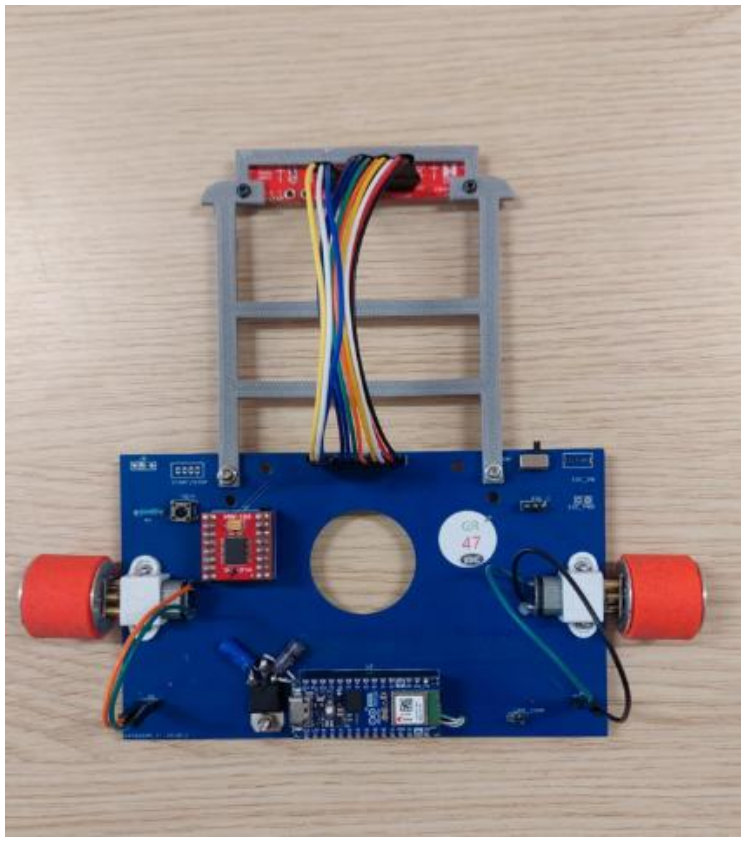
- **DC Gearmotors:** 3000rpm, 6V

- **TB6612FNG:** The TB6612FNG Motor Driver can control up to two DC motors at a constant current of 1.2A (3.2A peak). Two input signals (IN1 and IN2) can be used to control the motor in one of four function modes: CW, CCW, short-brake and stop. The two motor outputs (A and B) can be separately controlled, and the speed of each motor is controlled via a PWM input signal with a frequency up to 100kHz.

- **Arduino Nano ESP32:** An Arduino Nano with a Xtensa dual-core 32-bit LX6 microprocessor, operating at 240 MHz and an Ultra low power co-processor. It also has a 12-bit SAR ADC up to 18 channels for optimal signal processing.

- **QTR-8RC Reflectance Sensor Array**: This sensor module has 8 IR LED/phototransistor pairs mounted on a 0.375" pitch, making it a great detector for a line-following robot. Pairs of LEDs are arranged in series to halve current consumption, and a MOSFET allows the LEDs to be turned off for additional sensing or power-savings options. Each sensor provides **a separate digital I/O-measurable output.**

- **7.4V 800mAh LiPo Battery:** utilized to provide a reliable power source, ensuring sufficient energy for extended operational durations while maintaining optimal power management for the robot's components, and not weighting too much, as to interfere with the systems design.



3.3. Software Architecture

The software architecture of the robot was structured into three main modules:

1. **Sensor Data Acquisition:**
    - The IR sensors continuously monitor the reflectivity of the surface. The analog values are converted into a digital format using Arduino's internal Analog-to-Digital Converter (ADC) .
    - A preprocessing step normalizes the sensor readings to reduce noise and improve reliability.

2. **PD Computation:**
    - The PD algorithm calculates the control signal using the error derived from sensor inputs. The error represents the deviation of the robot's current position from the center of the line.
    - A time-based loop ensures consistent sampling intervals for accurate derivative and integral calculations.
    - Gain parameters (Kp, Kd) are defined as global variables and can be dynamically adjusted during runtime for fine-tuning.

3. **Motor Control:**
    - The calculated control signal is translated into motor speed adjustments. The left and right motors receive differential speeds to steer the robot back to the desired trajectory.
    - A PWM (Pulse Width Modulation) technique is used to control the motor speeds smoothly, ensuring gradual turns and acceleration.

3.4. Practical Implementation using Arduino IDE

On this part some essential parts of the code written in C are shown, such as sensor reading, PD calculations and motor control.

- **Sensor reading**

```
qtr.setTypeRC();
qtr.setSensorPins((const uint8_t[]){A7,A6,A5,A4,A3,A2,A1,9},
SensorCount);
qtr.setEmitterPin(2);
uint16_t position = qtr.readLineBlack(sensorValues);
```

- **PD Calculations**

  The code below determines the deviation from the line center and computes the control signal using proportional and derivative components in order to adjust the speed of each motor

```
int error = 3400 - position;
int deltaError = error - lastError;
int Pvalue = KP * error;
int Dvalue = KD * deltaError;
int motorSpeed = Pvalue + Dvalue;
lastError = error; // Operation for error prediction - derivative
term
totalError += error;
int rightMotorSpeed = DEFAULT_SPEED + motorSpeed;
int leftMotorSpeed = DEFAULT_SPEED - motorSpeed;
```

- **Motor Control**

  On this part of the code we control the polarity and speed of each motor accordingly from the PD calculations

```
digitalWrite(AIN1, HIGH);
digitalWrite(AIN2, LOW);
analogWrite(PWMA, abs(rightMotorSpeed));

digitalWrite(BIN1, HIGH);
digitalWrite(BIN2, LOW);
analogWrite(PWMB, abs(leftMotorSpeed));
```

4. Introduction

This study introduces a MATLAB-based simulation designed to evaluate the effectiveness of a Proportional-Derivative (PD) control strategy in guiding a robot along a predefined trajectory. By utilizing real-time feedback, the robot effectively minimizes deviations from the desired path. Path-following is a crucial capability for autonomous robots, requiring precise control mechanisms to minimize errors in trajectory tracking. In this work, a robot is tasked with following a trajectory represented by a sum of pseudo-random sinusoidal functions. A PD controller is employed to reduce tracking errors and maintain stability. It should be stated that this is a simulation, and, even though accurate, it does not take into account physical parameters, like friction, air resistance, sensor error and miss-input, and motor drift error, things that could hinder the robots ability to follow the linen and introduce a steady-state error in the real life application. Some key snippets of code are shown in this paper for better understanding of both the theory and the MATLAB interpretation.

The simulation aims to:

- Visualize the effect of control parameters on robot stability and responsiveness.
- Showcase the real-time dynamics of the robot and its motors.

5. System Model

5.1 Inputs to the System

The primary input is the **desired trajectory,** y_desired , is a pseudo-random representation of the desired path. In this sumulation y_desired consists of 3 sine waves, with random amplitudes, frequencies and phases. This input is used to simulate the input the robot would get in a real life scenario, from a sensor.

5.2 Output of the System

Robot Position (x, y): Updated dynamically and plotted against y_desired.

Motor Commands (Vleft, VrightV): Control signals applied to the left and right motors for steering and forward motion.

6. PD Controller Design

6.1 Tracking Error (e)

The error is the difference between the robot's current position and the desired trajectory:

$$e = y - y_{target}$$

The error represents the difference between the robot’s current position and the desired path.

6.2 Control Signal

The control signal u(t) combines proportional and derivative components:

$$u = -Kp * e - Kd * \left(\frac{de}{dt}\right)$$

```
% Calculate the error (distance from desired path)
   e = y - y_target;
   % Derivative of the error
   de_dt = (e - e_prev) / dt;
   % Control signal for PD controller
   control_signal = (-Kp * e - Kd * de_dt);
```

---

7. Motor Control (Voltage Calculation and Visualization)

The control signal u(t) is translated into motor voltages:

$$voltage_left = v - u \qquad\qquad voltage_right = v + u$$

Where:

- **v**: Base speed of the robot (constant), and predefined.
- **u**: Control signal.
- **Clamping:** Motor voltages are clamped between[0, 7] volts to simulate physical constraints.

```
% Motor voltages (simplified model for visualization)
   voltage_left = max(0, min(7, v - control_signal)); % Clamp to [0, 7]
   voltage_right = max(0, min(7, v + control_signal)); % Clamp to [0, 7]
```

For the motor voltages to be clearly understandable, and visually pleasing we have added 2 bars under the tracking simulation, indicating the voltages of the motors, and the difference between them. The bar graph dynamically represents the motor voltages. Some key takeaways are:

- Sharp turns: Significant difference between left and right motor voltages. (figures 1,2)
- Straight motion: Similar voltages for both motors. (figures 3,4)

Moreover**, an Amplification Factor** was introduced. The control signal is scaled by an amplification factor (multiplied with a number) to exaggerate the voltage differences for better visualization. This amplification is purely for visual clarity between the voltages of the 2 motors and does not indicate real values.

```
% Amplification factor for visual clarity
   amplify_factor = 4.5;
   % Apply additional amplification for visualization only
   visual_voltage_left = voltage_left * amplify_factor;
   visual_voltage_right = voltage_right * amplify_factor;
```

8. Real-Time Path Tracking

The robot's position and orientation are updated iteratively based on the control output:

1. **Orientation Update:** The control signal adjusts the robot's heading angle:

$$\theta(t) = \theta \cdot (t - \Delta t) + u(t) \cdot \Delta t$$

Where:

- **θ(t):** The robot's heading angle at time t, measured relative to a reference direction (e.g., forward).
- **u(t):** The control signal from the PD controller, which acts like a steering input.
- **Δt:** The time step for each update.

2. **Position Update:** The robot moves forward in the direction of its current orientation:

$$x = x + v \cdot cos(\theta) \cdot \Delta t$$

$$y = y + v \cdot sin(\theta) \cdot \Delta t$$

Where:

- **x(t), y(t):** The robot's position at time t.
- **v**: The constant forward speed of the robot.
- **cos(θ(t)):** The component of velocity in the horizontal (x-axis) direction.
- **sin(θ(t)):** The component of velocity in the vertical (y-axis) direction.
-

```
% Update robot orientation (steering angle) and positions
   theta = theta + control_signal * dt ;
   x = x + v * cos(theta) * dt;
   y = y + v * sin(theta) * dt;
```

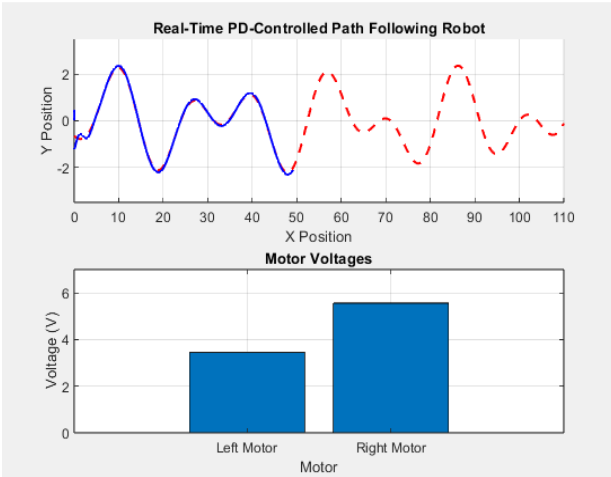


Fig.1: Simulation Robot taking a shard right turn

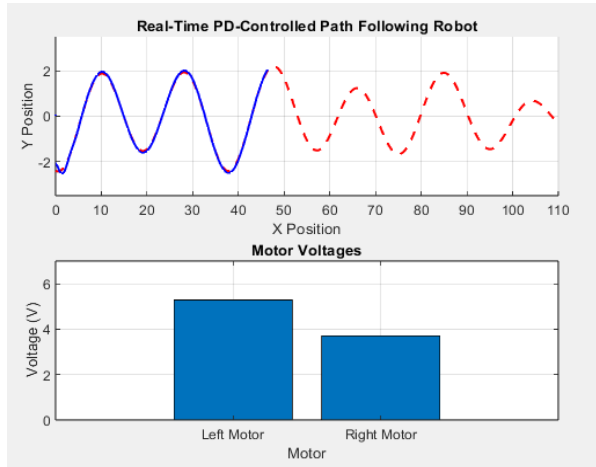


Fig.2 : Simulation Robot preparing to take a shard right turn

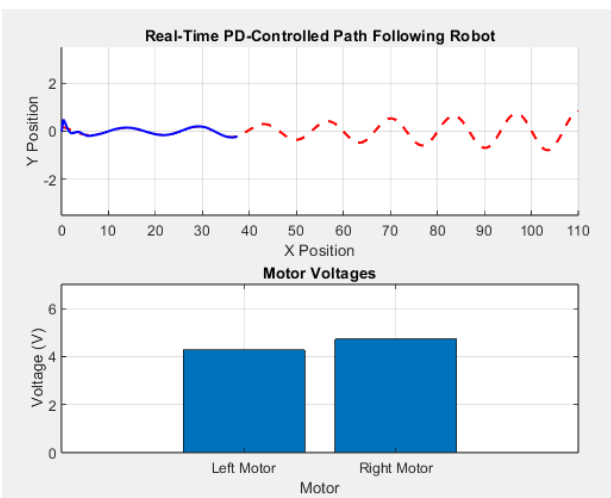


Fig.3 : Simulation Robot going in a semi-straight path

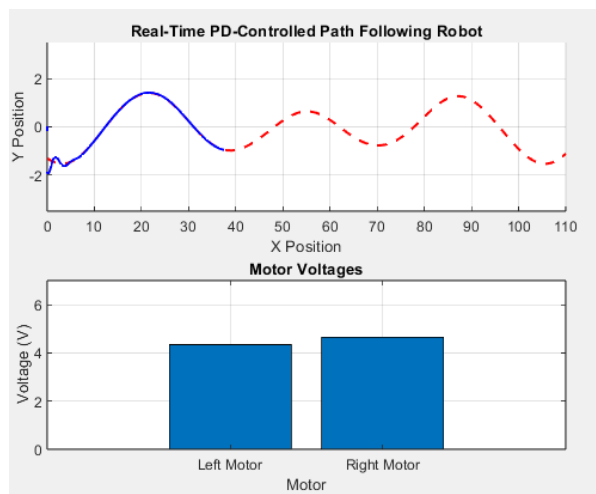


Fig.4: Simulation Robot taking a wide turn

Its is apparent, that in sharp turn scenarios the motor voltages are quite different from each other, cased by a bigger control signal. On the other hand, in wider turns, and straight lines, the control signal is not as big as in the previous case, so the motor voltages remain quite equal.

That is because of how motor voltages are calculated in this simulation, and in the real life application (see chapter 7, Motor Control)

9. System Stability and Tuning

The performance of the robot depends on careful tuning of **Kp** and **Kd**:

- **Under-damped System (low Kd):** Fast response but overshoots and oscillates around the path.
- **Over-damped System (high Kd):** Smooth response but slower corrections to errors.
- **Balanced Tuning:** Achieves a fast, stable response with minimal oscillation and good path tracking.

Similarly, the tuning of **Kp** affects the system:

- **Under-damped System (high Kp):** Fast response with high sensitivity to errors, but it can overshoot and oscillate.
- **Over-damped System (low Kp):** Slow response with reduced sensitivity to errors, resulting in sluggish corrections.
- **Balanced Tuning :** Achieves a proportional response with enough sensitivity to track the path accurately while avoiding overshooting and excessive oscillations.

Key Takeaways

- **PD Control:** Combines proportional and derivative feedback to guide the robot along the desired path with minimal error and oscillation.
- **Motor Control:** Translating control signals into motor voltages allows the robot to steer dynamically.
- **Real-Time Feedback:** Continuous updates based on error ensure the robot adapts to its current state and tracks the desired path effectively.
- **Practical Tuning:** Fine-tuning Kp and Kd is essential for achieving balance between responsiveness and stability.

10. References and Sources

- *The PID Controller Line Follower Robot with STM32F103 - Serhat Mizrak Abdullah - Gul University and Furkan Durmuş Abdullah - Gul University: https://www.researchgate.net/publication/377657580_The_PID_Controller_Line_Follower_Robot_with_STM32F103*

- *Design of a Control Approach to Assist the Performance of a Competitive Line Follower Robot - Johann S. J. C. C. Amorim, Jefferson H. O. Fernandes, Matheus S. Machado, Andre L. C. Canella, Milena F. Pinto ´ Federal Center for Technological Education of Rio de Janeiro*

- *Omega Engineering: https://www.omega.com/en-us/resources/pid-controllers*

- *Wikipedia: https://en.wikipedia.org/wiki/Proportional–integral–derivative_controller*

- *2 popular LLM were used (OpenAI's ChatGPT 4 and ChatGPT 3, and Microsoft's Copilot), for debugging, and solving problems with the simulation part of this paper (mainly on plotting the results)*

- *MATLAB's embedded tutorials were used for better understanding and programming of the simulation*